

Computer Architecture & Organization Fundamentals

Abstraction Layers, Architecture vs. Organization, Stored-Program Concept, and
Memory Models in AArch64

Computer Organization & Architecture | Topic 1

Course Information & Administration

Instructor Contact

- **Email:** jburns@ada.edu.az
- **Office:** B317
- **Office Hours:** Thursdays 11:00–15:00 (or by appointment)
- **Note:** Avoid Blackboard messages for urgent matters.

Classroom Expectations

- **The Golden Rule:** Once class commences and the instructor speaks, no outside conversations are permitted.
- **Authentic Work:** Emphasize the *why* alongside the *what* and *how*. Do not pass off AI-generated output as your own.

Tools, Emulation, and Assessments

Software & Emulation Stack

- **C & ARM64 Assembly:** Core languages utilized for architecture programming.
- **QEMU:** Full-system and user-space emulator for AArch64 virtual execution.
- **Compiler Explorer:** Live ARM64 assembly inspection from C source.

Online Programming Resources

- **WebAssembliss:** Browser-based ARM64 Linux assembly environment:
https://web.assembliss.app/editor/arm64_linux/
- **CPUlator:** Interactive web-based ARM32 / ARMv7 system simulator:
<https://cpulator.01xz.net/?sys=arm>

Topic 1 Outline

Foundational Focus

Understanding the bridge connecting high-level algorithms to the physical silicon execution substrate in modern 64-bit ARM systems.

- **Section 1:** The Full Stack of Abstraction Layers
- **Section 2:** Computer Architecture versus Computer Organization
- **Section 3:** Instruction Set Architecture (ISA) versus Microarchitecture
- **Section 4:** The Stored-Program Concept & Execution Cycle
- **Section 5:** Memory Models: von Neumann, Harvard, and Modified Harvard
- **Section 6:** Summary and Interactive Self-Test Review

Section 01

The Full Stack of Abstraction Layers

The 9 Levels of Computer Abstraction

Computers manage immense complexity by enforcing clean hierarchical interfaces:

Application Software	C Programs, Algorithms, Apps
Operating Systems	Device Drivers, Schedulers, Virtual Memory
Architecture (AArch64 ISA)	64-Bit Instructions, X Registers
Organization (Microarchitecture)	Datapaths, Controllers, Pipelines, Caches
Logic Design	Adders, ALUs, Register Files, Decoders
Digital Circuits	and, or, not, xor Logic Gates
Analog Circuits	Differential Amplifiers, Filters, Clocks
Electronic Devices	NMOS / PMOS Field-Effect Transistors
Solid-State Physics	Electron Transport in Silicon Substrates

Where Software Meets Hardware

CSCI 2406 Focus

Our course focuses on the critical boundary that translates logical intent into physical electrical actions: computer architecture and organization.

Above the Boundary (Architecture / Software)

- Applications compiled into machine binaries.
- Operating systems managing hardware through architectural exception levels.
- Completely portable across any hardware implementing the identical AArch64 ISA.

Below the Boundary (Organization / Hardware)

- Organization and microarchitecture (datapaths and state controllers).
- Digital circuits and gate-level logic.
- Silicon fabrication nodes and thermal design power (TDP) constraints.

Computer Architecture vs. Computer Organization

What is Computer Architecture?

Computer Architecture defines the **externally visible capabilities and behavior** of the machine—the *what*.

- The programmer-visible specification of the computer system.
- Establishes what the software is allowed to execute and what state changes occur.
- Determines the binary interface targeted by compilers and assembly programmers.

Architectural Specifications Include:

- The complete **Instruction Set Architecture (ISA)** vocabulary (AArch64).
- Programmer-visible registers (x0–x30, stack pointer sp, program counter pc, status flags).
- Instruction encodings, data formats (byte, halfword, word, doubleword), and addressing modes.

What is Computer Organization?

Computer Organization describes the **internal hardware implementation** of the architecture—the *how*.

- The physical arrangement of operational units and their logical interconnections.
- Defines how the machine physically performs the architectural operation.
- Completely transparent (invisible) to the programmer and machine software.

Organizational Structures Include:

- Datapaths, control units, ALUs, and multiplexer routing.
- Pipeline staging, branch prediction logic, and cache hierarchies (L1/L2/L3).
- Internal bus interfaces, clocking frequencies, and memory controller arbitration.

Architecture vs. Organization: Direct Contrast

Computer Architecture (<i>What</i>)	Computer Organization (<i>How</i>)
Programmer / Compiler view	Hardware engineer / Silicon designer view
Functional contract and behavior	Physical implementation and datapath
Changes slowly across generations	Evolves rapidly with silicon manufacturing nodes
Example: Does a 64-bit multiply exist?	Example: Is multiplication implemented via a hardware multiplier array or Booth recoder?
Specifies visible registers & flags	Specifies pipeline stages, ALU internal circuits
Software target (C, ARM64 assembly)	Hardware target (Verilog, VHDL, RTL)

Architectural Stability and Hardware Evolution

The Benefit of a Stable Architecture

Preserving the instruction set architecture allows software investments to persist indefinitely while hardware design is continuously overhauled to boost performance.

Architectural Invariance

- The **AArch64 ISA** defines standard instructions like `add`, `ldr`, and `cbz`.
- An ARM64 binary compiled for early AArch64 systems executes seamlessly on modern Apple M-series or AWS Graviton CPUs.

Organizational Evolution

- **Early Cores:** In-order or narrower out-of-order execution pipelines.
- **Modern Cores:** Wide-issue out-of-order engines, deep multi-level caches, and advanced execution units.

ISA versus Microarchitecture

The Instruction Set Architecture (ISA)

The **ISA** is the complete vocabulary of machine instructions supported by the hardware.

- All software running on a given processor must be expressed using its specific ISA.
- Different programs use different subsets, but all conform to the common ISA dictionary.

The Architectural Contract: add x1, x2, x3

The ISA guarantees a single semantic state change:

$$\text{Register } x_1 \leftarrow \text{Register } x_2 + \text{Register } x_3$$

- The architecture defines *what* state update occurs upon completion.
- It does not state *how* data moves across physical buses or wires inside the CPU.

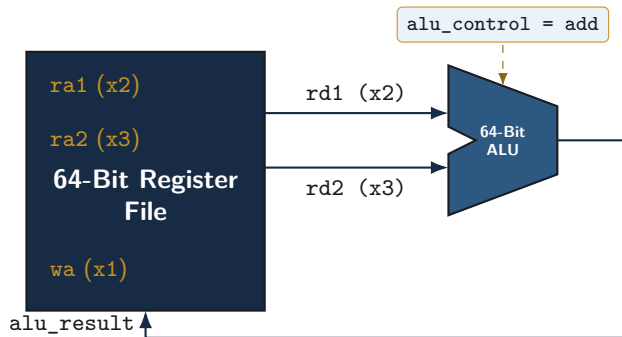
Microarchitecture: Executing add x1, x2, x3

Register Transfer Steps for add x1, x2, x3

- ❶ `ra1 <- x2; ra2 <- x3` *(Assert register read addresses)*
- ❷ `rd1 <- RegisterFile[ra1]; rd2 <- RegisterFile[ra2]` *(Read 64-bit operand values)*
- ❸ `alu_control <- add` *(Configure ALU operation multiplexers)*
- ❹ `alu_result <- rd1 + rd2` *(Execute 64-bit addition)*
- ❺ `reg_write <- 1` *(Assert register file write enable)*
- ❻ `RegisterFile[x1] <- alu_result` *(Commit result to destination register)*

- **ISA View:** Add 64-bit values in x2 and x3; store sum into x1.
- **Microarchitecture View:** Sequence control signals, bus transfers, and ALU operations.

Visualizing the Microarchitectural Datapath



- Microarchitectures differ by doing this sequentially in multiple cycles or parallelized within single/pipelined cycles.

One ISA — Multiple Hardware Realizations

Microarchitecture	Datapath Style	Design Objective
Single-Cycle	Every instruction takes 1 long clock cycle	Simple control; slow clock rate
Multi-Cycle	Instructions take variable number of short cycles	Low hardware cost; reuse functional units
Pipelined (Classic 5-Stage)	Overlap fetch, decode, execute, memory, writeback	Higher instruction throughput (1 inst/cycle ideal)
Superscalar Out-of-Order	Multiple pipelines, dynamic instruction scheduling	Maximum performance for desktops/servers

Key Principle

All four microarchitectures execute the identical compiled AArch64 binary accurately.

The Stored-Program Concept

The Stored-Program Concept

Foundational Definition

Program instructions are encoded as binary numbers and stored in the exact same addressable memory subsystem alongside standard runtime data.

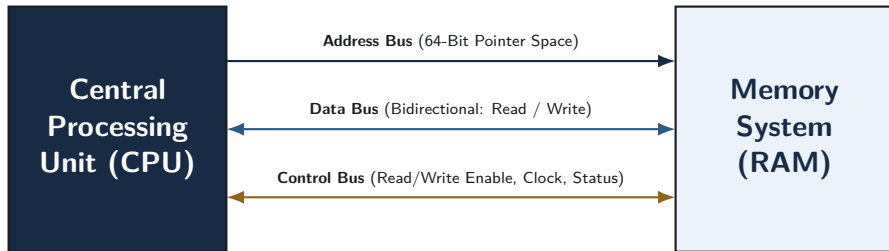
Capabilities of a General-Purpose Computer:

- **Store Information:** Holds data values and program sequences electronically.
- **Manipulate Information:** Performs mathematical and logical transformations via the ALU.
- **Make Decisions:** Alters control flow dynamically based on evaluated conditions (e.g., conditional branches).

Core Advantage

Reprogramming does not require physical rewiring; it simply requires loading new numbers into memory.

The CPU–Memory Interface: Three Fundamental Buses



- **Address Bus:** Selects the specific memory cell location using 64-bit addresses.
- **Data Bus:** Transfers the numerical word to or from the selected cell.
- **Control Bus:** Dictates the direction of transfer (read vs. write) and timing signals.

Memory as an Array of Numbered Cells

How Memory is Structured:

- Memory is viewed logically as a contiguous 1-D array of cells.
- Each cell has a unique numerical **Address** and stores a **Value**.
- Address 4 holds value 3.
- An address is simply an index; the data is the payload.

Address	Stored Value
0	7
1	3
2	15
3	20
4	3 ← Cell content
5	7
6	3

Context Determines Meaning

Bit Invariance Principle

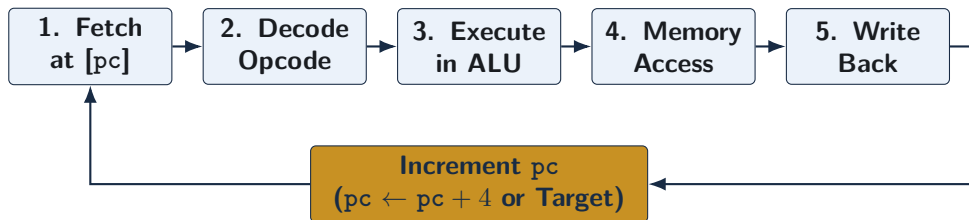
A pattern of bits residing in a memory cell possesses no intrinsic, permanent identity. Its meaning is determined entirely by **how the CPU accesses and interprets it**.

Consider a 32-bit word stored in memory at location 0x0x0000000000400000:

- If the **program counter (pc)** points here → the CPU fetches and decodes it as an **AArch64 instruction**.
- If an **arithmetic load instruction** reads this address → the CPU treats it as a **signed / unsigned integer**.
- If a **pointer variable** holds this address → it is treated as a **64-bit memory reference**.
- If passed to a **display routine** → it is rendered as **ASCII/UTF-8 text characters**.

The Classical Machine Instruction Lifecycle

The processor continuously steps through code using the **program counter (pc)**:



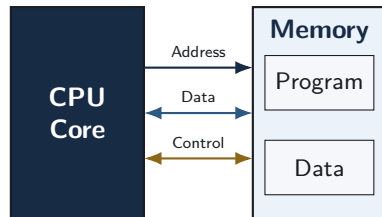
- Because code is in memory, programs can be loaded, copied, relocated, or modified dynamically.

von Neumann vs. Harvard Architectures

The von Neumann Architecture

Core Characteristics:

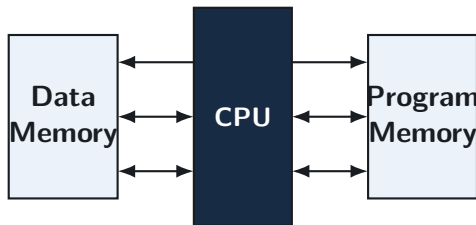
- **Unified Memory:** A single address space containing both instructions and data.
- **Shared Bus System:** A single set of address, data, and control lines is shared.
- **Operation:** The CPU fetches an instruction, then subsequently accesses data using the same bus.
- Directly realizes the stored-program concept in its purest form.



The Harvard Architecture

Core Characteristics:

- **Separated Memories:** Physically independent instruction and data memory modules.
- **Independent Buses:** Dedicated address, data, and control lines for each memory bank.
- **Parallel Operation:** CPU can fetch an instruction and read/write data operands **simultaneously**.
- Instruction and data words can have differing bit-widths and storage technologies.



Structural Separation: The Key Distinction

The Fundamental Question

The decisive architectural issue is not simply where bytes are stored. It is **whether instruction access and data access are structurally separated**.

Single Shared Bus (vN)

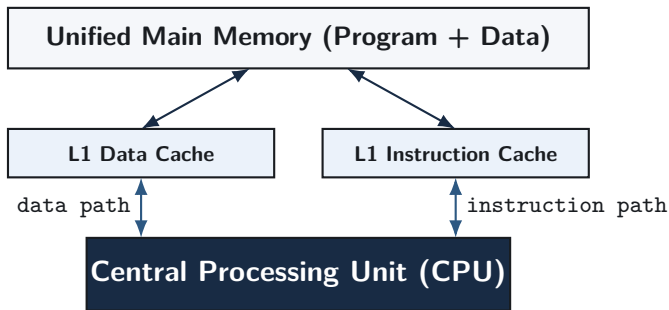
- Sequential access creates bus contention (**von Neumann bottleneck**).
- Lower hardware routing cost and flexible memory budgeting.

Dedicated Dual Buses (Harvard)

- Concurrent instruction fetch and operand read/write eliminates structural stalls.
- Higher pin counts and rigid memory partition limits.

The Modern Standard: Modified Harvard Architecture

Modern RISC systems (e.g., Apple M-series, ARM Cortex-A64) merge both concepts:



- **Internally:** Separate L1 caches yield Harvard pipeline throughput.
- **Externally:** Unified RAM preserves von Neumann software simplicity.

Comparison Summary: Memory Topologies

Model	Bus & Memory Structure	Primary Modern Use Case
von Neumann	Single shared memory and bus for instructions and data	Low-cost microcontrollers, simple embedded state controllers
Harvard	Physically separate memories and buses for code vs. data	Digital Signal Processors (DSPs), ATmega/AVR Arduino boards
Modified Harvard	Split L1 caches inside CPU; unified main memory outside	Modern high-performance RISC / CISC (ARM Cortex, Apple Silicon, x86-64)

Summary & Self-Test

Topic Summary

- **Architecture vs. Organization:** Architecture is the programmer-visible specification (the *what*); Organization is the internal hardware implementation (the *how*).
- **ISA vs. Microarchitecture:** ISA is the instruction vocabulary contract; Microarchitecture specifies the datapath and control sequencing.
- **Stored-Program Concept:** Instructions are stored as binary numbers in memory alongside data; meaning is determined by how the CPU accesses them.
- **von Neumann Architecture:** Shares a single memory and bus system for both instructions and data.
- **Harvard Architecture:** Uses separate memory modules and dedicated buses for concurrent instruction and data access.
- **Modified Harvard:** Blends separate L1 caches near the CPU core with unified main memory.

Self-Test: Questions 1 & 2

Question 1: Architecture vs. Organization

What is the difference between computer architecture and computer organization? Provide two specific features belonging to each domain.

Question 2: ISA vs. Microarchitecture

For the AArch64 instruction `add x1, x2, x3`, what does the ISA specify, and what internal actions belong to the microarchitecture?

Question 3: The Stored-Program Concept

What is the stored-program concept, and why was it essential for creating general-purpose computing machines?

Self-Test: Solutions 1 & 2

Solution 1

Architecture is the software-visible specification (e.g., 64-bit register count, ISA opcodes).

Organization is the hardware implementation (e.g., pipeline depth, cache associativity).

Solution 2

- **ISA Specifies:** Add contents of x2 to x3 and place the 64-bit result in x1.
- **Microarchitecture Executes:** Read registers x2 and x3 via port addresses ra1/ra2, configure ALU control to add, assert write-enable (reg_write=1), and commit to x1.

Self-Test: Questions 4 & 5

Question 4: Memory Model Topologies

What is the primary structural difference between von Neumann and Harvard architectures, and why can Harvard machines achieve higher cycle efficiency?

Topic 1 Self-Test Question 5: Modified Harvard Design

Explain what a Modified Harvard architecture is. How does Apple Silicon or ARM implement this concept in hardware?

Self-Test: Solutions 4 & 5

Solution 4

von Neumann uses a single unified bus and memory, causing serial access contention. Harvard uses independent buses and memories, allowing simultaneous instruction fetch and data operand load/store in the same clock cycle.

Solution 5

A Modified Harvard architecture provides separate L1 instruction and data caches directly adjacent to the CPU core to eliminate pipeline stalls, while utilizing a unified L2/L3 cache and main memory system to maintain a unified address space.